BEETTU
VML24CS079
S4 CSE-D ROLL NO:15

ASSIGNMENT-1

1.Write a C program priority.c that implements non-preemptive priority CPUscheduling (high value has higher priority).

Input Format:

n
PID1 AT1 BT1 PR1
PID2 AT2 BT2 PR2
.....
PIDn ATn BTn PRn

Sample Input:

5
P1 0 5 2
P2 0 4 5
P3 1 3 1
P4 2 3 4
P5 3 3 3

Sample Output:

Waiting Time:
P1 10
P2 0
P3 14
P4 2
P5 4

Turnaround Time:
P1 15
P2 4
P3 17
P4 5
P5 7

Average Waiting Time: 6.00Average
Turnaround Time: 9.60

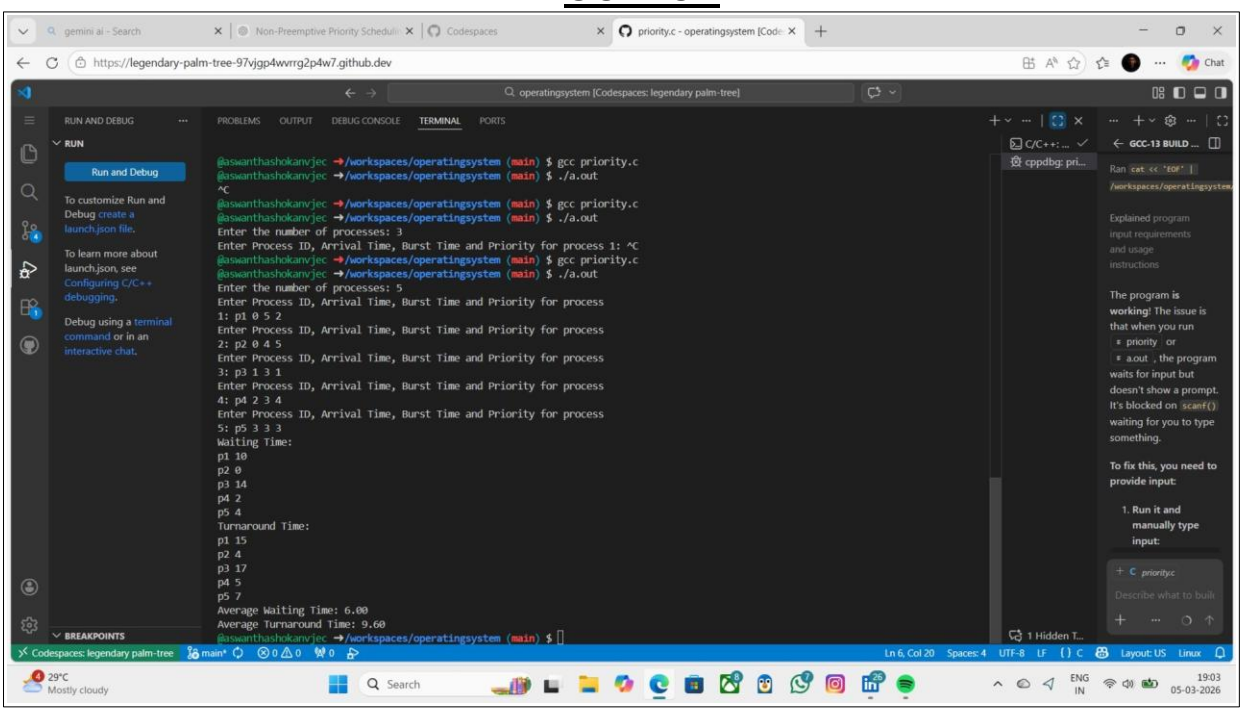
PROGRAM

[Preview] README.md | priority.c U | Settings | Extension: C/C++ Extension Pack

priority.c > main()

```c
#include <stdio.h>
#include <string.h>
struct process
{
    char pid[10];
    int at, bt, pr;
    int wt, tat, ct;
    int completed;
};
int main()
{
    int n, time = 0, completed = 0;
    float total_wt = 0, total_tat = 0;
    struct process p[20];
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
    {
        printf("Enter Process ID, Arrival Time, Burst Time and Priority for process \n%d: ", i + 1);
        scanf("%s %d %d %d", p[i].pid, &p[i].at, &p[i].bt, &p[i].pr);
        p[i].completed = 0;
    }
    while (completed < n)
    {
        int idx = -1;
        int highest = -1;
        for (int i = 0; i < n; i++)
        {
            if (p[i].at <= time && p[i].completed == 0)
            {
                if (p[i].pr > highest)
                {
                    highest = p[i].pr;
                    idx = i;
                }
            }
        }
        if (idx != -1)
        {
            time += p[idx].bt;
            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            total_wt += p[idx].wt;
            total_tat += p[idx].tat;
            p[idx].completed = 1;
            completed++;
        }
        else
        {
            time++;
        }
    }
    printf("Waiting Time:\n");
    for (int i = 0; i < n; i++)
        printf("%s %d\n", p[i].pid, p[i].wt);
    printf("Turnaround Time:\n");
    for (int i = 0; i < n; i++)
        printf("%s %d\n", p[i].pid, p[i].tat);
    printf("Average Waiting Time: %.2f\n", total_wt / n);
    printf("Average Turnaround Time: %.2f\n", total_tat / n);
    return 0;
}
```

OUTPUT




```
@aswanthashokanvjec ➜/workspaces/operatingsystem (main) $ gcc priority.c
@aswanthashokanvjec ➜/workspaces/operatingsystem (main) $ ./a.out
Enter the number of processes: 5
Enter Process ID, Arrival Time, Burst Time and Priority for process
1: p1 0 5 2
Enter Process ID, Arrival Time, Burst Time and Priority for process
2: p2 0 4 5
Enter Process ID, Arrival Time, Burst Time and Priority for process
3: p3 1 3 1
Enter Process ID, Arrival Time, Burst Time and Priority for process
4: p4 2 3 4
Enter Process ID, Arrival Time, Burst Time and Priority for process
5: p5 3 3 3
Waiting Time:
p1 10
p2 0
p3 14
p4 2
p5 4
Turnaround Time:
p1 15
p2 4
p3 17
p4 5
p5 7
Average Waiting Time: 6.00
Average Turnaround Time: 9.60
@aswanthashokanvjec ➜/workspaces/operatingsystem (main) $
```

GITHUB

https://github.com/Beettu/operatingSystem